# Compilation et empaquage Debian de librga-rockchip

https://gemini.google.com/app/88930af8a4066f6d

*User prompt: indiques moi les prerequis, librairies, dépendances nécessaire à la compilation de ce dépôt github et trouves moi les bugs erreurs à corriger pour compiler et créér un paquet deb https://github.com/Trak-unixbased/librga-rockchip*

Response: Le projet **librga** (accélérateur 2D matériel Rockchip) et ses forks (comme celui de *tsukumijima* ou le fork associé) utilisent le système de build **Meson / Ninja** (ou CMake selon la branche) avec une arborescence Debian standard (`debian/rules`, `debian/control`) pour générer les paquets `librga2` et `librga-dev`.

Voici le guide complet des prérequis, les points de blocage récurrents à corriger, et la procédure pas-à-pas pour compiler et générer les paquets `.deb`.

## 1. Prérequis et dépendances nécessaires

Installez l'outillage de compilation Debian, le moteur de build et les bibliothèques de développement requises (notamment `libdrm`) :

```
sudo apt update
sudo apt install -y \
    build-essential \
    debhelper \
    devscripts \
    dpkg-dev \
    git \
    meson \
    ninja-build \
    pkg-config \
    cmake \
    libdrm-dev
```

*Optionnel selon les sous-modules ou options activées :*

- `git` pour les hooks/versions.
- Si le projet est configuré avec support Wayland/DRM avancé : `libwayland-dev` ou les headers kernel Rockchip.

## 2. Bugs et pièges fréquents à corriger

Lors de la compilation ou du packaging de `librga-rockchip`, plusieurs erreurs surviennent classiquement :

### A. Détection ou version de `libdrm`

- **Symptôme :** `meson.build` échoue avec `Run-time dependency libdrm found: NO`.

- **Cause :** `pkg-config` ne trouve pas `libdrm.pc` ou le chemin standard multiarch (`/usr/lib/aarch64-linux-gnu/pkgconfig`) n'est pas interrogé.
- **Correction :** Assurez-vous d'avoir installé `libdrm-dev` et vérifiez avec `pkg-config --modversion libdrm`.

**B. Absence de commit / version dynamique dans `debian/rules`**

- **Symptôme :** Échec lors du calcul de la version par git si le dépôt est extrait via une archive zip (sans dossier `.git`) ou si `git describe` renvoie une erreur.
- **Correction :** Dans `debian/rules` ou `debian/changelog`, si la règle invoque `git rev-parse --short HEAD`, assurez-vous que le dépôt a bien son historique git, ou figez manuellement la version dans `debian/changelog` :

  ```
  dch -v 2.2.0-1 "Custom build for arm64"
  ```

**C. Chemins d'installation de bibliothèques multiarch (`libdir`)**

- **Symptôme :** `dpkg-shlibdeps` ou `dh_install` échoue parce qu'il attend les bibliothèques dans `/usr/lib/aarch64-linux-gnu` alors que Meson les installe par défaut dans `/usr/lib` (ou inversement).
- **Correction :** Dans `debian/rules`, assurez-vous que `dh_auto_configure` transmet bien `--libdir=lib/${DEB_HOST_MULTIARCH}` :

  ```
  override_dh_auto_configure:
  	dh_auto_configure -- -Dlibdir=lib/$(DEB_HOST_MULTIARCH)
  ```

  Et dans les fichiers `debian/librga2.install`, utilisez `usr/lib/*/librga.so.*` plutôt qu'un chemin codé en dur `usr/lib/librga.so.*`.

**D. Compatibilité C++14/C++17 et drapeaux stricts (`-Werror`)**

- **Symptôme :** Le compilateur s'arrête sur des avertissements liés aux casts de pointeurs vers les ioctls du noyau RGA (`rga.h`, structures DRM).
- **Correction :** Si la compilation s'arrête sur un warning, désactivez `werror` dans `meson.build` (`default_options: ['werror=false']`) ou dans `debian/rules` :

  ```
  dh_auto_configure -- -Dwerror=false
  ```

## 3. Procédure de compilation et création du paquet `.deb`

**Étape 1 : Cloner le dépôt avec ses sous-modules**

```
git clone --recursive https://github.com/Trak-unixbased/librga-rockchip.git
cd librga-rockchip
```

**Étape 2 : Vérifier les dépendances déclarées**

Avant de lancer la création du paquet, vérifiez qu'aucune dépendance système ne manque :

```
dpkg-checkbuilddeps
```

*(Si des paquets manquants sont listés, installez-les via `sudo apt install`.)*

**Étape 3 : Lancer la création des paquets Debian**

Compilez et empaquetez sans signer les clés GPG :

```
dpkg-buildpackage -us -uc -b
```

**Étape 4 : Vérification et installation des paquets générés**

Les paquets sont créés dans le répertoire parent (`../`) :

```
ls -lh ../librga*.deb
```

Vous obtiendrez généralement :

- `librga2_<version>_<arch>.deb` (bibliothèque partagée runtime)
- `librga-dev_<version>_<arch>.deb` (en-têtes C/C++ et fichier `.pc` pkg-config)

Pour tester l'installation :

```
sudo dpkg -i ../librga2_*.deb ../librga-dev_*.deb
```

**Sources**


1. dependency() - The Meson Build system
2. Built-in options - The Meson Build system
3. rockchip rga和mpp库编译原创 - CSDN博客


---